ATIVIDADE PRÁTICA

BACKEND DE CATEGORIAS

Projeto 2 (Laravel MVC) e Projeto 3 (API Node/Express). Você vai criar a área de Categorias espelhando o caminho que já existe para produtos.

01 O QUE VOCÊ VAI ENTREGAR

PARTE	PROJETO	VOCÊ CRIA
A	2 — Laravel MVC	Rotas, `CategoriaController` (index/show) e as views `categorias/index` e `categorias/show`
B	3 — API Node/Express	Endpoints `GET /api/categorias` e `GET /api/categorias/:id`

02 PARTE A — PROJETO 2 (LARAVEL MVC)

Espelhe o que já existe para produtos: `ProdutoController`, `routes/web.php` e as views em `resources/views/produtos/`.

> **Já pronto no frontend:** no topo das páginas existe um menu com *Produtos* e *Categorias*. O link *Categorias* aponta para a rota `/categorias` que você vai criar — antes disso, clicar nele dá erro 404, e isso é o esperado.

TAREFAS

1. **Rotas** em `routes/web.php`:
   - `GET /categorias` para a lista de categorias.
   - `GET /categorias/{id}` para o detalhe de uma categoria.
2. **Controller**: crie `app/Http/Controllers/CategoriaController.php` com:
   - `index()`: busca `Categoria::orderBy('id')->get()` e retorna a view `categorias.index`.
   - `show($id)`: busca `Categoria::findOrFail($id)` e os produtos com `Produto::where('categoria', $categoria->nome)->get()`, retornando `categorias.show`.
3. **Views**: crie em `resources/views/categorias/`:
   - `index.blade.php`: lista as categorias, reaproveitando `@extends('layouts.app')`.
   - `show.blade.php`: mostra os dados da categoria e a lista de produtos dela.

COMO TESTAR

```
docker compose up --build mvc
# abra: http://localhost:8092/categorias
```

Com as rotas, o controller e as views no lugar, o link *Categorias* do menu passa a abrir a listagem, cada card leva ao detalhe em `/categorias/{id}` e `/categorias/999` devolve 404 pelo `findOrFail`.

03 PARTE B — PROJETO 3 (API NODE/EXPRESS)

Crie os endpoints de categorias no mesmo estilo dos de produtos, em `03-api-react/api/src/server.js`.

> **Já pronto no frontend:** as telas React de categorias existem e funcionam — `Nav`, `CategoryList` (lista) e `CategoryDetail` (detalhe com os produtos da categoria).
>
> Elas já chamam os dois endpoints deste item. Enquanto a API não responder, a tela mostra uma mensagem de erro; assim que os endpoints existirem, a navegação passa a funcionar sozinha, sem editar nada no React.

CONTRATO DOS ENDPOINTS

MÉTODO E ROTA	RESPOSTA ESPERADA
`GET /api/categorias`	200 e um array com as categorias.
`GET /api/categorias/:id`	200 e a categoria com seus produtos. 404 se não existir.

EXEMPLO DE RESPOSTA DE GET /API/CATEGORIAS/1

```
{
  "id": 1,
  "nome": "Periféricos",
  "descricao": "Teclados, mouses e outros acessórios...",
  "produtos": [
    { "id": 1, "nome": "Teclado Mecânico RGB", "categoria": "Periféricos", ... },
    { "id": 2, "nome": "Mouse Gamer 16000 DPI", "categoria": "Periféricos", ... }
  ]
}
```

TAREFAS

1. Importe os dados no topo do `server.js`. O de produtos já está importado:

   ```
   import { categorias } from "./categorias.js";
   ```

2. Crie `GET /api/categorias` devolvendo o array `categorias`.

3. Crie `GET /api/categorias/:id`:
   - ache a categoria pelo `id`;
   - se não existir, responda 404 com `{ "error": "Categoria não encontrada" }`;
   - se existir, monte a resposta com a categoria mais os produtos dela: `produtos.filter(p => p.categoria === categoria.nome)`.

COMO TESTAR

```
docker compose up --build api

curl http://localhost:8093/api/categorias
curl http://localhost:8093/api/categorias/1
curl -i http://localhost:8093/api/categorias/999
```

Depois teste pelo frontend, em http://localhost:8094, clicando em *Categorias*:

- a listagem mostra as 5 categorias no lugar da mensagem de erro;
- clicar em uma categoria abre o detalhe com nome, descrição e os produtos dela;
- um id inexistente na URL cai na mensagem de "não encontrada" vinda do 404 da API.

04 COMO RODAR E RECONSTRUIR

```
# subir tudo
docker compose up --build

# reconstruir apenas um serviço depois de editar
docker compose up --build mvc   # Projeto 2 (Laravel)
docker compose up --build api   # Projeto 3 (API)

# ver os logs da API
docker compose logs -f api

# parar tudo
docker compose down
```